

Table of contents

Programmation Génétique (Genetic Programming - GP)	1
Introduction	1
Que doit-on attendre d'un tel programme ?	2
Régression Symbolique (Symbolic Regression - SR)	2
But de la régression symbolique	2
Pourquoi la SR est puissante	3
Inférence de Comportement (Behavior Inference)	3
Codage des Programmes	3
Calcul de la Fitness : Régression Symbolique	4
Codage des Programmes avec des S-expressions	4
Codage par Arbre (Tree-based Coding)	5
Ensemble de Fonctions (Function Set) et Ensemble de Terminaux (Terminal Set)	6
Propriété de Clôture (Closure Property)	7
Population Initiale	8
Crossover (à la Koza)	8
Mutation (à la Koza)	8
Exemple : Recherche de Modèles de Trading Optimaux	9
Exemple de Stratégie de Trading Obtenue	9
GP Linéaires Basés sur une Pile (Stack-based Linear GPs)	9
GP Linéaires	9
Crossover pour les GP Linéaires	10
Mutation pour les GP Linéaires	10
Exemple de GP Linéaire	11
Résultats pour une Exécution	11
Exemple : Fonction Arithmétique $F(x_0, x_1) = (x_0 + x_1)^2$	12
Contrôle de Flux (Flow Control)	13
Boucles (Loop)	14

Programmation Génétique (Genetic Programming - GP)

Introduction

La **Programmation Génétique** (Genetic Programming - GP) a été introduite et développée par John Koza au début des années 1990.

L'objectif est de :

- Appliquer les concepts évolutionnistes à des programmes informatiques plutôt qu'à un ensemble de valeurs numériques.

- Faire évoluer un programme pour qu'il puisse résoudre un problème donné de manière optimale.
- Ouvrir la voie à la robotique évolutionnaire.

Pour ce faire :

- On considère une population de programmes et on applique une sélection et une recombinaison (incluant le crossover) pour obtenir le programme désiré.
- Au lieu d'optimiser des paramètres numériques, la GP fait évoluer les programmes eux-mêmes jusqu'à ce qu'ils réalisent la tâche souhaitée.

Que doit-on attendre d'un tel programme ?

Il doit produire une sortie correcte pour toute entrée possible.

Il doit découvrir la fonction à partir des données de sortie correspondant aux données d'entrée.

Capable de réaliser : l'ajustement de courbes algébriques, la régression symbolique, des tâches de classification, etc.

Applicable à : la conception de modèles de trading en finance, la détermination de conditions aux limites à partir de données empiriques, la détermination du mouvement de robots dans des environnements complexes, etc.

Ceci est typique d'un processus d'apprentissage automatique (Machine Learning - ML).

Mais cela peut aussi être vu comme une tâche d'optimisation : minimiser la différence entre la sortie spécifiée et celle obtenue par la GP.

Cela montre la frontière ténue entre ML et optimisation... C'est maintenant un sujet qui appartient à l'IA et au ML.

Régression Symbolique (Symbolic Regression - SR)

But de la régression symbolique

- Construire un programme qui relie les données d'entrée aux données de sortie sans pré-définir la forme fonctionnelle.
- La SR explore le vaste espace de toutes les expressions symboliques possibles construites à partir d'un ensemble de fonctions et de variables.
- Un premier ensemble de données : l'ensemble d'apprentissage (training set) composé de données d'entrée et de sortie, est sélectionné pour générer des programmes.

- Le but est qu'un programme construit sur cet ensemble d'apprentissage soit capable de généralisation, c'est-à-dire qu'il ait détecté la relation sous-jacente entre les données d'entrée et de sortie.
- Cette généralisation est testée avec un autre ensemble de données, l'ensemble de test (test set), qui ne fait PAS partie de l'ensemble d'apprentissage.
- Il y a un risque de sur-ajustement (overfitting) : un excellent ajustement sur les données de l'ensemble d'apprentissage mais un résultat (très) médiocre avec l'ensemble de test.

Pourquoi la SR est puissante

- Aucune structure de modèle prédéfinie - La GP peut découvrir : des polynômes, des expressions rationnelles, des exponentielles, des compositions imbriquées, des modèles hybrides mélangeant des termes algébriques et transcendants.
- Interprétabilité : Contrairement aux réseaux de neurones ou aux forêts aléatoires, la GP produit des formules explicites et lisibles par l'homme.
- Construction automatique de caractéristiques (feature construction) : La GP découvre des expressions intermédiaires utiles (sous-arbres), créant effectivement de nouvelles caractéristiques.
- Flexibilité et non-linéarité : La GP explore naturellement des mappings non linéaires, produisant parfois des modèles comparables aux approches d'apprentissage automatique.

Inférence de Comportement (Behavior Inference)

- Lorsque le but est de déterminer le comportement d'un robot (ou d'un groupe de robots) à partir de sa perception de l'environnement.
- On peut simuler le comportement induit par la GP et voir comment il se comporte dans des environnements générés aléatoirement.
- Taux de succès ou d'échec pour les tâches attendues.
- Apprentissage par Renforcement (Reinforcement Learning - RL).

Codage des Programmes

Défi clé : le codage des programmes. Comment coder un programme pour appliquer des opérateurs évolutionnistes, tout en générant des individus valides (c'est-à-dire des programmes syntaxiquement corrects et exécutables) ?

Deux approches distinctes sont introduites et considérées :

- Les arbres (Trees)
- Les programmes linéaires basés sur une pile (Stack-based linear programs)

Dans tous les cas, l'espace de recherche (c'est-à-dire l'espace de tous les programmes possibles) est limité par la nature des langages de programmation informatique existants.

Calcul de la Fitness : Régression Symbolique

Soit p un programme tel que

$$y = p(x)$$

est la sortie générée par p avec l'entrée x . Notez que x et y peuvent être des structures de données (pas seulement des valeurs numériques).

Soit $A = \{\langle x_1, y_1 \rangle, \dots, \langle x_k, y_k \rangle\}$ un **ensemble d'apprentissage** (training set) composé de k paires $\langle \text{entrée}, \text{sortie} \rangle$.

La fitness f (ou fonction de perte) du programme p est définie comme :

$$f(p) = \sum_{i=1}^k d(y_i - p(x_i)) = \sum_{i=1}^k |y_i - p(x_i)|$$

C'est la capacité du programme p à générer la sortie attendue en fonction de l'entrée donnée.

Dans la majorité des cas de GP, le but est d'avoir une **représentation algébrique** d'une expression booléenne ou arithmétique.

Par exemple, on peut avoir une série d'observations $\{(x_i, y_i)\}$ et on cherche une relation $y = g(x)$ où g est donnée algébriquement par $g(x) = 2x^2 - 3x + 1$ ou $g(x) = x \sin x + 2$.

Note importante : Essayer de faire évoluer des programmes écrits en C, Java, Python, etc., en utilisant des opérations de crossover et de mutation est sans espoir !

La plupart des recombinaisons produiraient des programmes qui ne sont pas syntaxiquement valides.

Codage des Programmes avec des S-expressions

Koza a proposé de :

Coder ces programmes avec un langage fonctionnel (similaire à Lisp) construit sur des instructions appelées S-expressions (expressions symboliques).

Par exemple, $(+x2)$ signifie $x + 2$.

Basé sur ces concepts, l'expression $2x^2 + 3x + 1$ devient

$$(+(*(*xx)2)(+(*3x)1))$$

Si on remplace un opérateur dans une S-expression par une autre S-expression, on s'attend à une commande valide.

On peut donc définir des opérateurs génétiques (crossover et mutation) sur les S-expressions.

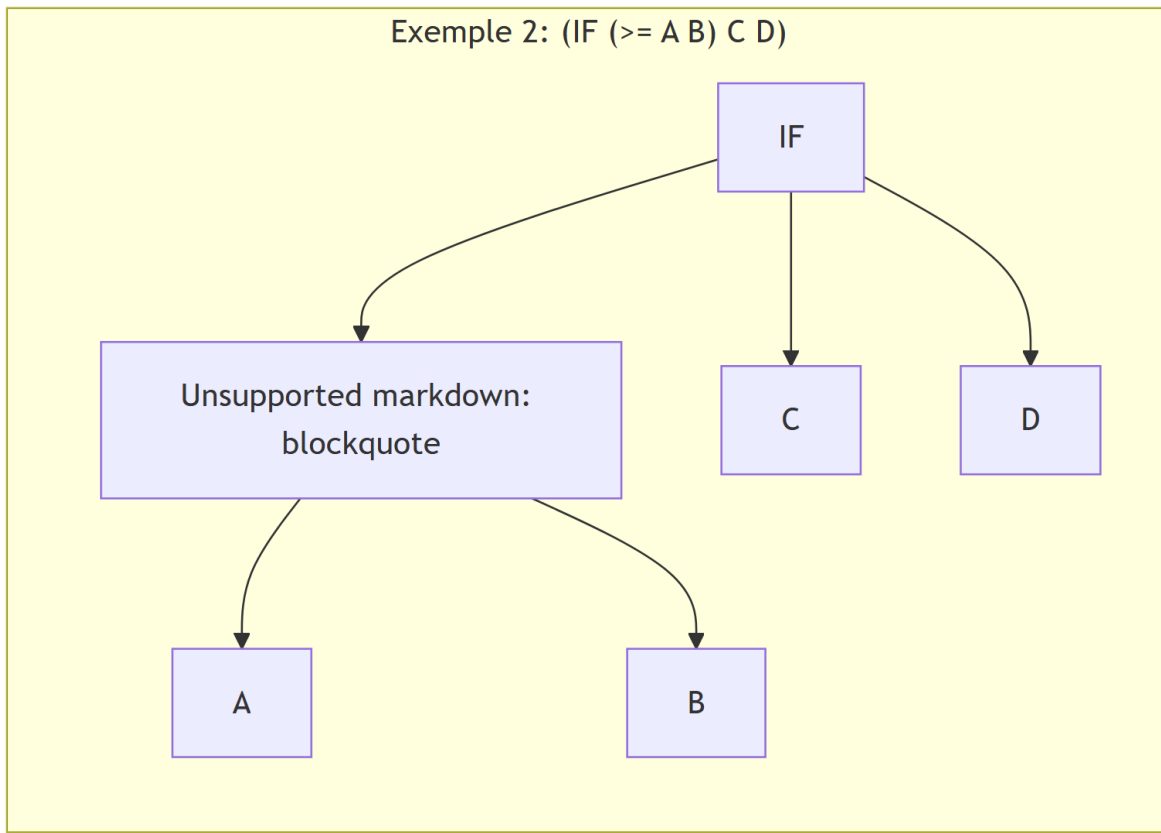
Note : Plusieurs S-expressions peuvent représenter la même fonction. Par exemple : $2x^2 + 3x + 1$ peut aussi être exprimée comme :

$$(+1(+(*2(*xx))(*3x)))$$

Codage par Arbre (Tree-based Coding)

C'est équivalent à une S-expression, mais plus facilement compréhensible.

Voici deux exemples :



Ensemble de Fonctions (Function Set) et Ensemble de Terminaux (Terminal Set)

Formellement, l'espace de recherche des GP est spécifié par un ensemble F de fonctions et un ensemble T de terminaux.

- F contient toutes les opérations possibles dans cet espace.
- T contient toutes les variables et constantes possibles.

Par exemple, on peut avoir l'ensemble de fonctions suivant :

$$F = \{+, -, *, /\}$$

et l'ensemble de terminaux suivant :

$$T = \{A, B, C, D\},$$

où A , B , C et D sont les noms des variables qui prennent des valeurs spécifiées pour chaque instance dans l'ensemble de test.

Pour les fonctions booléennes :

$$F = \{\text{AND, OR, NO}\}$$

avec des terminaux qui pourraient inclure des booléens et des constantes :

$$T = \{b_1, b_2, b_3, \dots, \text{TRUE, FALSE}\}.$$

Opérateur IF

On peut définir des opérateurs IF dans l'**ensemble de fonctions**.

Par exemple, (If A B C D) est une fonction de 4 arguments :

- Si $A=B$ alors C est retourné.
- Sinon D est retourné.
- D'autres formes plus simples de IF peuvent être considérées.

Propriété de Clôture (Closure Property)

Chaque fonction doit être capable d'accepter comme argument :

- Toutes les valeurs retournées par d'autres fonctions.
- Toutes les valeurs de l'ensemble de terminaux T .

La division, les opérateurs unaires et certains autres opérateurs peuvent avoir besoin d'être augmentés pour accepter n'importe quelle paire d'arguments.

L'espace de tous les programmes admissibles est constitué de toutes les compositions de fonctions disponibles de F avec des éléments terminaux de T .

Cet espace a potentiellement une dimension infinie et la taille des individus est arbitraire.

Il apparaît pratiquement nécessaire de limiter leur taille.

Certains morceaux de programmes peuvent ne rien faire : par exemple, $(-xx)$ et peuvent donc être simplifiés par la constante 0.

C'est le problème dit de "gonflement" (bloat).

La propriété de clôture est une contrainte fondamentale pour la GP.

Exemples :

- Division par zéro.
- Incompatibilité de type (Type mismatch).
- Erreurs de domaine (Domain errors).
- Structures de contrôle : (if x y z) avec x numérique et non booléen.

Population Initiale

En partant de l'ensemble de fonctions et de l'ensemble de terminaux, on peut générer une population initiale de programmes valides.

- Chaque nœud est tiré aléatoirement de F ou T .
- Si c'est un terminal, alors la branche se termine.
- Sinon, il est nécessaire de tirer autant d'éléments que requis par l'opérateur F .

On peut biaiser le choix entre F et T en fonction de la profondeur :

- Vers la racine, il y aura une plus grande probabilité de choisir une fonction, puis une plus grande probabilité de tirer un terminal.
- Ces probabilités définissent la profondeur moyenne des programmes.

Crossover (à la Koza)

L'opération de crossover peut être réalisée en choisissant aléatoirement un lien dans l'arbre représentant chaque parent et en échangeant les branches inférieures correspondantes.

Les points de crossover sont typiquement choisis avec une probabilité non uniforme **pour favoriser les nœuds plus internes** par opposition aux nœuds des feuilles et branches basses.

Mutation (à la Koza)

L'opérateur de mutation est effectué en sélectionnant aléatoirement un sous-arbre du programme, et en le remplaçant par un autre sous-arbre.

Le sous-arbre de substitution est généré aléatoirement à partir de F et T .

Exemple : Recherche de Modèles de Trading Optimaux

But : Faire évoluer un programme qui formulerait des recommandations d'investissement sur le marché des changes (forex).

Nous avons à notre disposition des indicateurs de marché $\{I_k(t)\}$ $k = 1, \dots, m$ qui sont typiquement des moyennes mobiles des taux de change à différentes échelles de temps.

Nous cherchons un programme qui prend les $\{I_k(t)\}$ comme entrées et qui produit trois sorties possibles : 0 (ne rien faire), -1 (vendre), et 1 (acheter).

La fitness est définie comme le rendement moyen généré par ce modèle s'il avait été utilisé au cours des années passées. À cette valeur, on soustrait le risque associé mesuré par sa variance.

Exemple de Stratégie de Trading Obtenue

Dans cet arbre, I_1 , I_2 , I_3 et I_4 sont des indicateurs complexes basés sur les moyennes mobiles de longues séries temporelles du forex. Les valeurs numériques sont des constantes empiriques.

Le rendement est bon, mais pas exceptionnel...

GP Linéaires Basés sur une Pile (Stack-based Linear GPs)

Pour suivre l'approche des langages procéduraux : coder les GP avec une série d'instructions exécutées séquentiellement.

Cependant, pour garantir la validité syntaxique du programme après crossover/mutation, on peut utiliser une approche basée sur une pile.

GP Linéaires

Par exemple :

- L'instruction A place la valeur de la variable A dans la pile.
- L'instruction B place la valeur de la variable B dans la pile.
- L'instruction ADD retire les deux éléments les plus hauts de la pile, et y ajoute un nouvel élément qui est la somme.

Ainsi le programme

```
A B ADD C MUL 2 sqrt DIV
```

correspond à

$$\frac{(A + B) \times C}{\sqrt{2}}$$

Contraintes

Les opérations doivent être adaptées pour tolérer le manque d'argument dans la pile.

L'espace des programmes est donc défini par l'ensemble de toutes les instructions possibles qui contiennent les variables, constantes et opérations.

Crossover pour les GP Linéaires

On peut utiliser le crossover de manière similaire à ce qui est fait avec une chaîne de bits.

Considérons 2 programmes et notons le point de crossover par le symbole |.

P1: A B ADD | C MUL 2 sqrt DIV

P2: 1 B SUB | A A MUL ADD 3

On obtient donc les deux programmes descendants suivants :

P1': A B ADD A A MUL ADD 3

P2': 1 B SUB C MUL 2 sqrt DIV

- P1' laisse dans la pile 2 valeurs : $A + B + A^2$ et 3.
- P2' laisse dans la pile 1 seule valeur : $(1 - B) * C / (\sqrt{2})$.

Mutation pour les GP Linéaires

Une mutation peut être réalisée en changeant aléatoirement une instruction par une autre.

Considérons 1 programme :

P1: A B ADD C MUL 2 sqrt DIV

On effectue une mutation sur la fonction ADD :

P1': A B MUL C MUL 2 sqrt DIV

P1' laisse dans la pile 2 valeurs : $(A * B) * C / \sqrt{2}$.

Exemple de GP Linéaire

Nous cherchons l'**expression algébrique** de la fonction booléenne f de 3 variables x_0, x_1, x_2 donnée par la table :

x_0	x_1	x_2	$f(x_0, x_1, x_2)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

La solution est évidente à l'œil humain :

$$f(x_0, x_1, x_2) = x_0 \text{ AND } x_1 \text{ AND } x_2$$

Nous considérons l'ensemble de terminaux :

$$T = \{X_0, X_1, X_2\}$$

et l'ensemble de fonctions :

$$F = \{\text{AND}, \text{OR}, \text{NOT}\}$$

Considérons un programme avec 5 instructions. Avec 6 choix par instruction, on peut construire $6^5 = 7,776$ programmes (recherche exhaustive).

La sortie est le sommet de la pile après exécution.

Résultats pour une Exécution

Considérons une population de 12 programmes de longueur 5, que nous faisons évoluer sur 50 générations (sélection, crossover, mutation).

La fitness compte combien de lignes dans la table sont correctement calculées. Ainsi, la fitness optimale est 8.

Population initiale :

- ['OR', 'NOT', 'X2', 'OR', 'X0'] fitness= 5
- ['X1', 'X2', 'OR', 'X0', 'OR'] fitness= 2
- ['OR', 'X2', 'OR', 'X0', 'X0'] fitness= 5
- ['OR', 'X2', 'X2', 'X1', 'X1'] fitness= 5
- ['X2', 'X0', 'X0', 'OR', 'X0'] fitness= 5
- ['OR', 'OR', 'AND', 'X1', 'X0'] fitness= 5
- ['X0', 'X0', 'X1', 'X2', 'X0'] fitness= 5
- ['X0', 'OR', 'X2', 'X0', 'OR'] fitness= 3
- ['AND', 'X2', 'NOT', 'OR', 'X0'] fitness= 5
- ['OR', 'AND', 'X2', 'X1', 'AND'] fitness= 7
- ['X0', 'AND', 'AND', 'X0', 'NOT'] fitness= 3
- ['X2', 'X2', 'X2', 'X0', 'X0'] fitness= 5

Population finale :

- ['X2', 'OR', 'X1', 'X0', 'AND'] fitness= 7
- ['X2', 'X2', 'NOT', 'X2', 'AND'] fitness= 7
- ['X1', 'X2', 'AND', 'X0', 'AND'] fitness= 8
- ['X2', 'OR', 'X1', 'X0', 'AND'] fitness= 7
- ['X2', 'OR', 'X1', 'X0', 'AND'] fitness= 7
- ['X2', 'OR', 'X1', 'X0', 'AND'] fitness= 7
- ['X2', 'OR', 'X0', 'OR', 'AND'] fitness= 3
- ['X2', 'OR', 'NOT', 'X2', 'AND'] fitness= 7
- ['X2', 'OR', 'AND', 'X0', 'AND'] fitness= 7
- ['X2', 'OR', 'AND', 'X0', 'AND'] fitness= 7
- ['X2', 'X0', 'X0', 'OR', 'AND'] fitness= 7
- ['X2', 'OR', 'NOT', 'X2', 'AND'] fitness= 7

NB : L'optimum n'est pas systématiquement trouvé à chaque exécution. La fonction (x0 and x1) or x2 est intéressante car plus difficile à trouver puisque x2 est un attracteur de fitness 7.

Exemple : Fonction Arithmétique $F(x_0, x_1) = (x_0 + x_1)^2$

Soit $A = \text{TrainingSet} = [(x_0^{(1)}, x_1^{(1)}, F(x_0^{(1)}, x_1^{(1)})) \dots]$.

Minimiser la fitness = $\sum_{(x_0, x_1) \in A} |F(x_0, x_1) - p(x_0, x_1)|$.

Ensemble de fonctions : ["ADD", "MUL", "DUP", "SUB"].

Ensemble de terminaux : ["X0", "X1"].

Ensemble d'apprentissage : [(1, 1, 4), (1, 3, 16), (1, 1, 4), (1, 2, 9), (2, 1, 9), (2, 3, 25), (2, 1, 9), (2, 2, 16), (3, 1, 16), (3, 3, 36), (3, 1, 16), (3, 2, 25), (4, 1, 25), (4, 3, 49), (4, 1, 25), (4, 2, 36)].

Population initiale (15 individus) :

- ['X1', 'X1', 'MUL', 'X0', 'X1'] fitness= 34
- ['MUL', 'DUP', 'X1', 'MUL', 'X1'] fitness= 34
- ['X0', 'X1', 'X0', 'X1', 'X1'] fitness= 34
- ['X0', 'X0', 'X1', 'ADD', 'X0'] fitness= 32
- ['X1', 'X0', 'X1', 'X0', 'DUP'] fitness= 32
- ['X0', 'X1', 'DUP', 'X0', 'X1'] fitness= 34
- ['X1', 'DUP', 'X1', 'X0', 'X1'] fitness= 34
- ['X0', 'X1', 'X1', 'X1', 'MUL'] fitness= 32
- ['DUP', 'X1', 'X1', 'DUP', 'X1'] fitness= 34
- ['ADD', 'DUP', 'ADD', 'MUL', 'DUP'] fitness= 1600* (pénalité, le programme ne fait rien)
- ['X0', 'X1', 'X1', 'X1', 'X1'] fitness= 34
- ['X0', 'X1', 'X1', 'X1', 'ADD'] fitness= 32
- ['MUL', 'X1', 'DUP', 'X1', 'X0'] fitness= 32
- ['X1', 'X1', 'ADD', 'DUP', 'MUL'] fitness= 20
- ['X1', 'X1', 'MUL', 'X0', 'X0'] fitness= 32

Population finale (100 générations) :

- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X1', 'ADD', 'DUP', 'MUL'] fitness= 20
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X1', 'ADD', 'DUP', 'MUL'] fitness= 20
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0
- ['X1', 'X0', 'ADD', 'DUP', 'MUL'] fitness= 0

Contrôle de Flux (Flow Control)

On peut aussi définir des structures de contrôle telles que des instructions conditionnelles IF ou des boucles.

On peut imaginer que l'instruction IF prend l'élément du sommet α de la pile et le compare à 0.

- Si $\alpha \geq 0$, les instructions qui suivent IF sont normalement exécutées.
- Si $\alpha < 0$, les instructions cessent d'être exécutées jusqu'à ce qu'une instruction ENDIF soit rencontrée.

⇒ Il est donc nécessaire d'ajouter un drapeau 0 ou 1 qui indique si les instructions doivent être exécutées ou non.

Boucles (Loop)

Nous avons la pile de données et une nouvelle pile : la pile de contrôle pour les boucles.

- Cette pile de contrôle contient la position de l'instruction LOOP dans le programme et le nombre d'itérations, pris du sommet de la pile de données.
- Dès que END-LOOP est rencontré, on décrémente le nombre d'itérations et on retourne à la position de LOOP, sauf si le nombre d'itérations est NULL. Dans ce cas, on retire le sommet de la pile de contrôle.
- Nous travaillons avec le sommet de la pile de contrôle, de sorte que des boucles imbriquées sont possibles.
- Cette construction de programme est robuste même lorsque les paires LOOP-END-LOOP ne sont pas équilibrées ou que le nombre d'itérations n'est pas défini.

Exemple

Nous considérons le programme suivant : Étant donné une valeur d'entrée X0 et un compte de répétition X1, calculer :

$$\text{somme} = X0 + X0 + \dots + X0 \text{ (X1 fois)}$$

En mathématiques : $\text{somme} = X0 \times X1$.

Avec GP : [X1, LOOP, X0, ADD, END-LOOP] avec 2 piles : pile de données et pile de contrôle.

Comment cela fonctionne-t-il ? Supposons les entrées X0=3 et X1=2.

Nous attendons 3+3=6.

- Instruction 1 : Empiler X1 (2). Pile de données : [2] & pile de contrôle : [].
- Instruction 2 : LOOP : Pile de données : [] & pile de contrôle : [(2, position_de_LOOP)]
→ le programme itérera son corps 2 fois.
- Instruction 3 : Empiler X0 (3) dans la pile de données : [3].

- Instruction 4 : ADD : opérandes insuffisantes → initialiser l'accumulateur et dans la pile de données : [3].
- Instruction 5 : END-LOOP :
 - Regarde le sommet de la pile de contrôle : (restant = 1, start_position = LOOP_index).
 - Décrémente restant : maintenant 1.
 - Puisque restant > 0, sauter au début de LOOP.
 - Pile de contrôle : [(1, LOOP_index)].
 - Pile de données inchangée : [3].
- Deuxième itération :
 - Instruction 3 (à nouveau) : Empiler X0 (3) dans la pile de données : [3, 3].
 - Instruction 4 : ADD : dans la pile de données : [6].
 - Instruction 5 : END-LOOP :
 - * Regarde le sommet de la pile de contrôle : (restant = 0, start_position = LOOP_index) et retire (0, LOOP_index) de la pile de contrôle.
- Programme terminé : la sortie est ce qui reste dans la pile de données : [6].